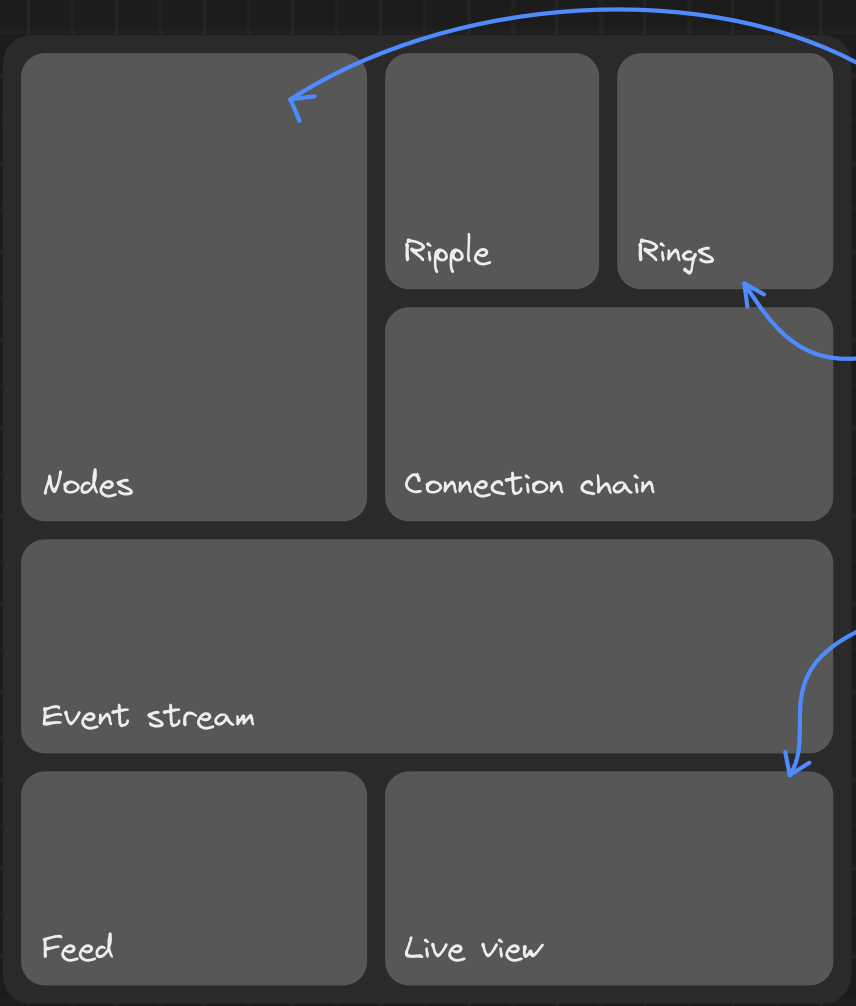


Nodes first

Every provider is a point with an identity, a place and a state. The biggest block, because everything starts here.

Ripple discovery

Search 500 m, then 1 km, then 2 km, until enough agents answer.

Events, not drawings

Agents only emit events. The Grid turns them into state, the map only renders state.

Example

Where agents meet the map.

Grid Pattern

A technical blueprint of the Streetwise Grid method: registry, ripple discovery, agent connection chain, event stream and live view, and how to build it for everyone.

what is inside

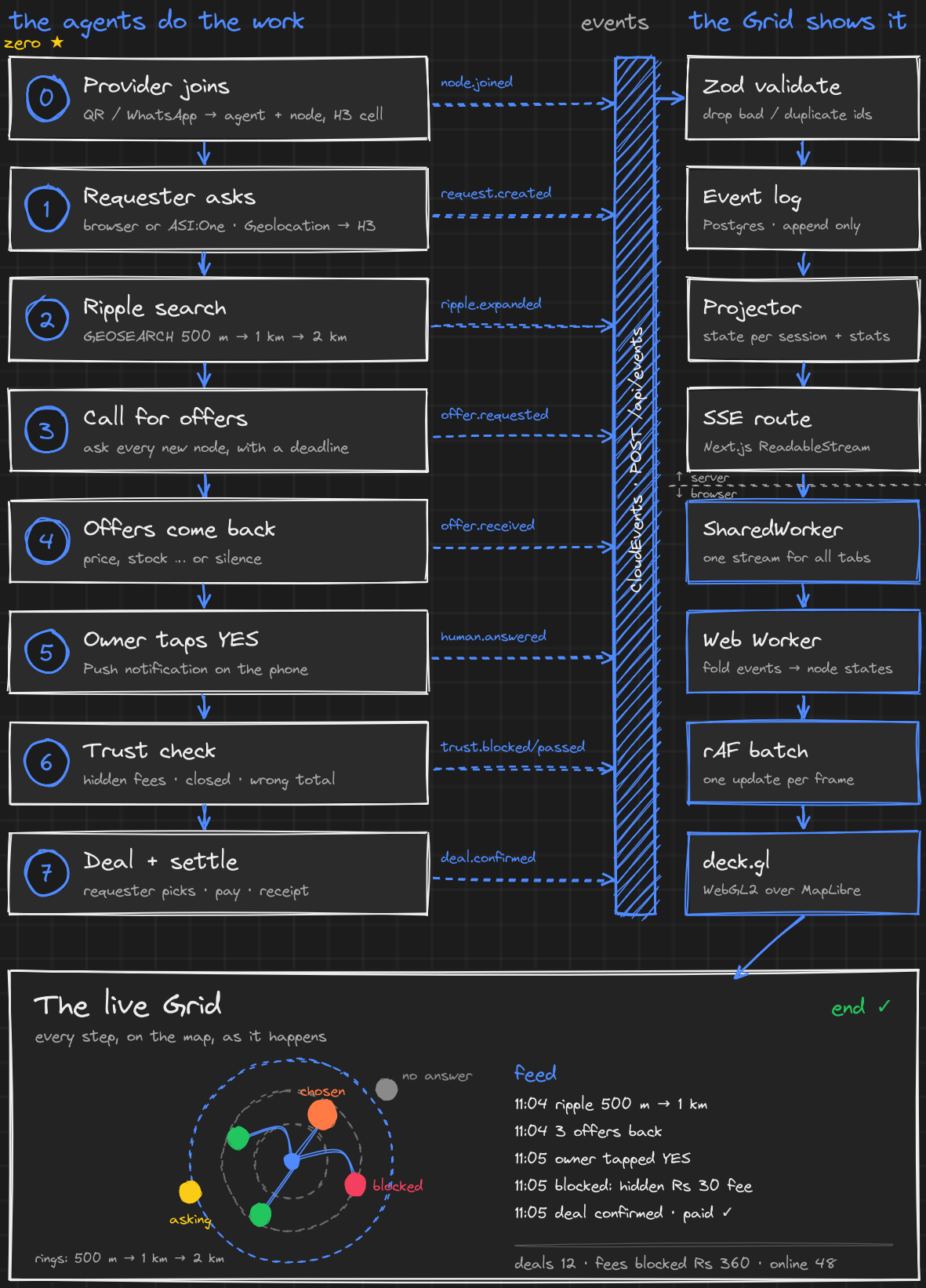
Contents

0. How it works, zero to end
1. The Grid method in one page
2. What exists today and its limits
3. Reference architecture
4. Node registry and identity
5. Spatial index and ripple discovery
6. The connection protocol
7. Event model
8. State projection
9. Live transport to viewers
10. Rendering and browser tech
11. Privacy and security
12. Reliability and scale
13. One Grid, many domains
14. Is it new? Where it stands
15. Build plan, step by step
16. Making it usable by everyone

References

the whole flow, hand-drawn ↘

How it works, from zero to end



the whole idea ↘

1. The Grid method in one page

The Streetwise Grid is a live view of a network of agents placed on a map. Strip away groceries, payments and the LLM, and the method is made of five parts. Everything in this document is about building those five parts well.

Part	What it means	In Streetwise today
1. Nodes	Every provider is a point with an identity, a location and a current state	`REGISTRY` of shops, each with its own Merchant Agent
2. Ripple discovery	Search a small circle first, widen until enough nodes answer	500 m → 1 km → 2 km until 3 shops are found
3. Connection chain	Agents message each other: ask → offer → check → accept → settle	QuoteRequest, owner YES/NO, Trust Agent, order, FET payment
4. Event stream	Every step of the chain emits a small event	`grid.searched` / `blocked` / `ordered` / `paid`
5. Live view	Events turn into node states and animations on a map, plus a feed	Leaflet map, rings, coloured pins, event list

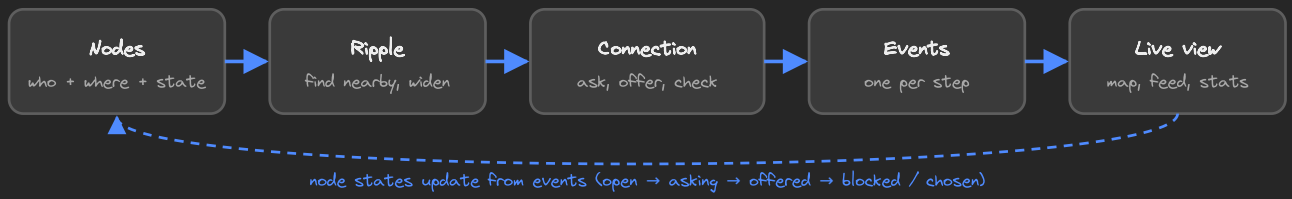


Figure 1. The five parts of the Grid method and the feedback loop between them.

note

The core idea: agents never draw anything. They only emit events. A separate Grid service turns events into state, and viewers only render state. This separation is what lets the same Grid show shops today and blood donors, EV chargers or tractors tomorrow.

where we are ↘

2. What exists today and its limits

`grid.py` keeps four module-level variables (`LAST`, `BLOCKED`, `CHOSEN`, `EVENTS`). Agents call helper functions in the same process; `/api/grid` builds a snapshot; the page polls it every 3 seconds and redraws everything. It is a good hackathon design, but it has clear limits.

Limit	Effect	Fix (section)

Global state, one "last search"	Two requesters at once overwrite each other; `searched()` clears blocks	Per-session state from events (§8)
Agents must share one process	Cannot run agents on different machines or by other builders	Event bus + standard envelope (§3, §7)
Poll every 3 s, full redraw	Up to 3 s delay, flicker, tooltips jump	SSE push + diff updates (§9, §10)
Memory only, 30 events	Restart loses everything, no replay	Append-only event log (§8, §12)
Linear scan of `REGISTRY`	Fine for 50 shops, slow for 50 000	Spatial index (§5)
Missing steps	Quotes, owner waiting and the ring widening are not shown	Full event vocabulary (§7)
No auth, exact locations	Anyone on the Wi-Fi sees orders and locations	Roles, signed events, location fuzzing (§11)
Hard-coded start view (Gurgaon)	Wrong place for any other city	Fit to data (§10)

the big picture ↘

3. Reference architecture

The target design has six components. Each one can start tiny (a Python module, SQLite) and grow later without changing the others.

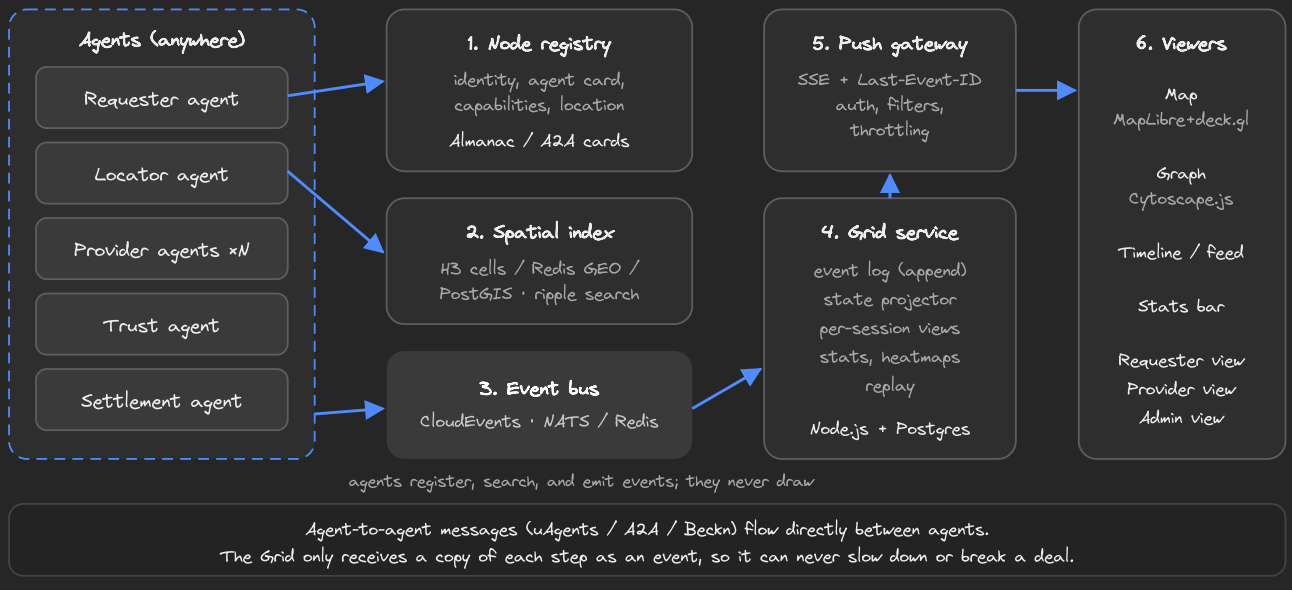


Figure 2. Reference architecture of a Grid connection network.

Component	Start with (hackathon)	Grow to (production)
1. Node registry	`nodes.json` + Fetch.ai Almanac, served by a Next.js route	Registry API (Node.js) with signed agent cards, verification status, expiry

2. Spatial index	`h3-js` cells in a `Map`	Redis GEO (`ioredis`) for live nodes; Postgres + PostGIS for history
3. Event bus	Node `EventEmitter` in one process	NATS JetStream (`nats.js`) or Redis Streams, subjects per domain and city
4. Grid service	SQLite (`better-sqlite3`) append-only table	Postgres via Drizzle or Prisma (ClickHouse for analytics), projectors as Node workers
5. Push gateway	Next.js / Hono route returning an SSE `ReadableStream`	Several gateway instances fanned out with Redis pub/sub, behind HTTP/2
6. Viewers	React + MapLibre GL, markers updated in place	deck.gl layers in a Web Worker-fed view, graph and timeline views, `<grid-map>` web component

Full-stack web stack (one language end to end)

The agents can stay in Python (uAgents); everything around the Grid can be TypeScript, so one developer can own it all.

Layer	Recommended	Why
Language	TypeScript everywhere, shared `@grid/schema` package (Zod)	Same event types validated on server and in the browser
App / API	Next.js (App Router route handlers) or Hono on Node / Bun	SSE streams, REST and pages in one deploy
Data	Postgres + PostGIS, Drizzle ORM, Redis	Spatial SQL, typed queries, fast live lookups
Bridge to agents	Python agents POST CloudEvents to `/api/events` or publish to NATS	Agents stay in uAgents, Grid stays in TS
Front end	React, MapLibre GL JS, deck.gl, TanStack Query, Tailwind	GPU maps, cached snapshots, quick UI
Deploy	Docker Compose locally; Vercel / Fly.io / a VPS for long-lived SSE	SSE needs long-lived connections, so pick a host that allows them

who is on the grid ↘

4. Node registry and identity

A Grid is only as good as its list of nodes. Every node needs a stable ID, a way to reach its agent, what it can do, and where it is. Do not invent this from zero; three existing designs already solve most of it.

Existing design	What to borrow
Fetch.ai Almanac	On-chain register of agents with endpoints (weighted), protocol digests and time-limited registrations that must be renewed. Expired entries drop out, which keeps the Grid free of dead nodes.
A2A Agent Card (Linux Foundation)	A JSON file at `/.well-known/agent-card.json` describing name, skills, capabilities and endpoints; v1.0 adds signed cards for identity and integrity.

Beckn / ONDC registry	A network registry of subscribers (buyer apps, seller apps, gateways) with public keys, so every message can be signed and verified.

Recommended node record

```
{
  "node_id":   "agent1q...",                  // agent address = identity
  "kind":      "provider",                    // provider | requester | service
  "domain":    "grocery",                     // grocery, health, ev, labour ...
  "name":      "Sharma Kirana",
  "location":  {"h3_r9": "893da1...", "lat": 28.4672, "lng": 77.0710, "precision_m": 50},
  "capabilities": ["quote", "confirm", "pay_agent_charge"],
  "hours":     "Mo-Su 07:00-22:00",           // OpenStreetMap opening_hours format
  "card_url":  "https://.../.well-known/agent-card.json",
  "verified":  {"gst": true, "phone": true},
  "expires_at":"2026-12-31T00:00:00Z"         // renew or drop off the Grid
}
```

find them fast ↘

5. Spatial index and ripple discovery

The ripple (500 m → 1 km → 2 km) is the most recognisable part of the Grid. To make it fast at any scale, keep nodes in a spatial index instead of scanning the whole registry.

Option	How it works	Best for	Query
H3 (Uber)	Earth split into hexagons at 16 resolutions (0–15). Each cell is a 64-bit ID; neighbours are computed, not stored. Uber uses H3 for pricing and dispatch.	Ripples, heatmaps, privacy (share a cell, not a point), aggregation	`grid_disk(cell, k)`
Redis GEO	Sorted set with geohash scores; `GEOSEARCH` (Redis 6.2+) by radius or box, nearest first, with a count limit	Live "who is near and online" in milliseconds	`GEOSEARCH ... BYRADIUS 500 m ASC COUNT 10`
PostGIS	Spatial SQL with R-tree style (GiST) indexes	History, analytics, joins with orders and ratings	`ST_DWithin(geog, point, 500)`

remember

Recommendation: use H3 as the common language of the Grid (every node and event carries an H3 cell), Redis GEO for the live search, and PostGIS later for history. H3 resolution 9 cells average about 0.1 km², roughly a city block, which fits shop-level discovery well; resolution 7 or 8 suits city-wide heatmaps.

Ripple algorithm (general form)

```
// ripple.ts  (npm i ioredis)
import Redis from "ioredis";
const redis = new Redis(process.env.REDIS_URL!);

export async function ripple(c: { lat: number; lng: number }, need: Need, session: string,
                             rings = [500, 1000, 2000], want = 3, deadlineMs = 20_000) {
  const asked = new Set<string>();
  let offers: Offer[] = [];
  for (const radius of rings) {
    await emit("ripple.expanded", session, { ...c, radius_m: radius });
    // GEOSEARCH key FROMLONLAT lng lat BYRADIUS r m ASC COUNT 25
    const ids = (await redis.geosearch("grid:nodes", "FROMLONLAT", c.lng, c.lat,
                                       "BYRADIUS", radius, "m", "ASC", "COUNT", 25)) as string[];
    const fresh = ids.filter(id => !asked.has(id));
    fresh.forEach(id => asked.add(id));
    offers = offers.concat(await callForOffers(fresh, need, deadlineMs));   // section 6
    if (offers.filter(isGood).length >= want) break;
  }
  return offers;   // fewer than wanted: show "no more nodes nearby" on the Grid
}

// H3 alternative (npm i h3-js): latLngToCell(lat, lng, 9) then gridDisk(cell, k)
```

Improvements over today: only ask new nodes in each ring, give every ring a deadline, and emit an event for each ring so the Grid can animate the circle growing. Rings can also adapt: in a dense market start at 200 m, in a village at 5 km (pick the first ring so that it is expected to contain about *want* × 2 nodes, using the H3 density of past nodes).

how agents talk ↘

6. The connection protocol

The message chain behind the Grid is a modern version of the Contract Net Protocol (Reid G. Smith, 1980), later standardised by FIPA: a manager sends a *call for proposals* with a deadline, participants propose or refuse, the manager accepts one, and the contractor reports completion. India's Beckn / ONDC uses the same shape with asynchronous callbacks.

Step	Contract Net (FIPA)	Beckn / ONDC	Streetwise	Grid event
Find	—	search → on_search (via gateway)	ShopSearch → ShopList	`ripple.expanded`, `node.found`
Ask	cfp	select	QuoteRequest	`offer.requested`
Answer	propose / refuse	on_select	QuoteList	`offer.received` / `offer.declined`
Human check	—	—	Owner YES / NO	`human.waiting` → `human.answered`
Trust	—	—	CheckQuotes → Verdict	`trust.blocked` / `trust.passed`

Commit	accept-proposal	init → confirm	OrderRequest → OrderResult	deal.confirmed
Done	inform-done	status, track	Payment, receipt	settle.paid , deal.done

Two lessons from the 1980 paper still apply: always use deadlines (a silent node must not block the chain) and only ask relevant nodes (message overhead grows with every node you ask). On the Grid, a node that misses its deadline should turn grey with a "no answer" label, which is useful information in itself.

one format for every step ↘

7. Event model

Use the CloudEvents envelope (a CNCF graduated standard since January 2024, used by Azure Event Grid, Google Eventarc, Knative and others). It gives every event the same outer shape (`id` , `source` , `type` , `specversion` , `time` , `subject`), so any agent from any builder can feed the Grid.

```
{
  "specversion": "1.0",
  "id":       "01JB7X...",                  // unique, used for de-duplication
  "source":   "agent1a...dukaan-sharma",    // which agent emitted it
  "type":     "grid.offer.received",
  "subject":  "session/7f3a",               // groups events of one request
  "time":     "2026-10-17T11:04:09Z",
  "data": {
    "node_id": "agent1a...", "h3": "893da1...",
    "summary": "Rs 186 for 3 items",        // short text for the feed
    "trace_id": "4bf92f3577b34da6...",      // link to OpenTelemetry trace
    "visibility": "session"                 // public | session | admin
  }
}
```

Event vocabulary

Type	Meaning	Grid effect
`grid.node.joined / left`	Node registered, renewed or expired	Pin appears / fades
`grid.node.status`	Open, closed, busy, out of stock	open closed
`grid.request.created`	A requester asked for something	Pulse at the centre
`grid.ripple.expanded`	Search ring widened	Circle grows
`grid.offer.requested`	Requester asked a node	Arc goes out
`grid.offer.received / declined / timeout`	Node answered or stayed silent	Arc comes back offered
`grid.human.waiting / answered`	Waiting for a person to tap YES / NO	asking pulsing
`grid.trust.blocked / passed`	Trust agent verdict with reason	blocked
`grid.deal.confirmed / done / cancelled`	Deal lifecycle	chosen

`grid.settle.paid`	Payment or agent charge, with tx link	Coin animation, feed link
`grid.redirect`	Closed node (e.g. QR scan) points to an open one	Arrow between nodes

Link events to agent traces. OpenTelemetry has semantic conventions for generative AI, including agent spans (still marked *Development*). If each event carries a `trace_id`, clicking a pin can open the full agent trace (LLM call, tool calls, messages) in tools such as Jaeger, Langfuse or similar. That turns the Grid from a picture into a debugger.

events become state ↘

8. State projection

Store events in an append-only log and compute state from them (event sourcing). The current state of every node in a session is just a fold over its events:

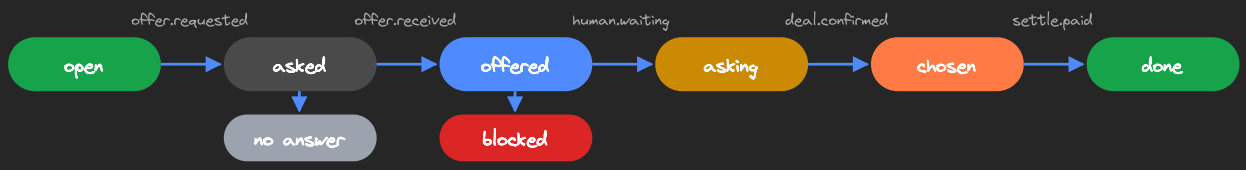


Figure 3. Node state machine inside one session. The base state (open / closed) comes from `node.status`.

- Per-session views: state is keyed by `(session, node)`, so two requesters never overwrite each other. The public city view shows only `node.status` and anonymous counts.
- Snapshot + deltas: on connect a viewer gets one snapshot (with the last event ID), then only new events.
- Replay for free: re-running the log at 1× or 5× speed gives a demo mode that never depends on live APIs.
- Stats are projections too: deals done, ₹ of hidden fees blocked, median time to first offer, nodes online per H3 cell.

push, don't poll ↘

9. Live transport to viewers

Option	Strengths	Weaknesses	Use for
Polling (today)	Simplest	Delay, wasted requests, full redraws	Fallback only
Server-Sent Events	Plain HTTP, one-way server → browser, automatic reconnect; the browser sends `Last-Event-ID` so the server can resend what was missed; multiplexed over HTTP/2	One-way; text only; HTTP/1.1 limits connections per domain	The Grid (recommended)

WebSocket	Two-way, binary	Resume after reconnect must be built by hand; more infrastructure	Owner YES / NO, chat

```
// app/api/grid/stream/route.ts  (Next.js App Router, Node runtime)
export const runtime = "nodejs";
export const dynamic = "force-dynamic";

export async function GET(req: Request) {
  const last = Number(req.headers.get("last-event-id") ?? 0);
  const view = await viewFor(req);                          // auth: public | session | admin
  const enc = new TextEncoder();
  const stream = new ReadableStream({
    async start(ctrl) {
      for await (const ev of log.follow({ after: last, view, signal: req.signal })) {  // replay, then live
        ctrl.enqueue(enc.encode(`id: ${ev.seq}\nevent: ${ev.type}\ndata: ${JSON.stringify(ev)}\n\n`));
      }
      ctrl.close();
    },
  });
  return new Response(stream, { headers: {
    "Content-Type": "text/event-stream", "Cache-Control": "no-cache, no-transform",
    "Connection": "keep-alive" } });
}

// browser (React hook)
useEffect(() => {
  const es = new EventSource(`/api/grid/stream?session=${id}`);
  es.addEventListener("grid.offer.received", e => apply(JSON.parse(e.data)));
  return () => es.close();
}, [id]);
```

Throttle on the server, not the browser: when a city has thousands of events per second, send node state changes at most every 250 ms per viewer and aggregate the rest into H3 counts. Scale out by running several gateways that each subscribe to the bus (or to Redis pub/sub) and serve their own viewers.

the browser side ↘

10. Rendering and browser tech

Map stack

Library	Role	When
Leaflet (today)	Simple raster map with markers	Up to a few hundred nodes
MapLibre GL JS	Open-source vector maps, smooth zoom, dark styles without CSS filters	Base map for production
deck.gl (from Uber)	GPU layers over MapLibre for very large data (millions of points)	Animations and scale
Cytoscape.js / Sigma.js	Network graph of agents and messages	Graph view

deck.gl layers that map directly to the Grid

- ScatterplotLayer: nodes, coloured by state, radius by importance.
- ArcLayer: `offer.requested` and `offer.received` as arcs between requester and node; fade after a few seconds.
- TripsLayer: animated trails for anything that moves (delivery riders, ambulances, trucks).
- H3HexagonLayer: demand heatmap and "no provider here" gaps, using the H3 cell on every event.
- Circle / polygon layer: ripple rings that grow with `ripple.expanded`.

Visual grammar (keep it the same in every domain)

Thing	Shape / motion	Colour
Available node	Solid dot	green
Unavailable / no answer	Hollow dot, faded	grey
Message out / back	Arc with moving dash	blue
Waiting for a human	Slow pulse	amber
Blocked by trust	Cross + reason on hover	red
Chosen / deal	Larger dot + ring	orange
Payment	Short coin flash, link in feed	gold

Also: fit the view to the data instead of a fixed city, keep labels off by default except for nodes in the active session, respect `prefers-reduced-motion`, never use colour alone (add icons and text), and keep the feed readable as text for screen readers.

Browser technology that makes the Grid fast

Most of the Grid's work can happen in the browser. The server only streams events; the browser projects, caches and draws them.

Browser API	Use in the Grid
EventSource (SSE)	One live stream per viewer, automatic reconnect with `Last-Event-ID`
SharedWorker or BroadcastChannel	Open the stream once and share it across tabs, instead of one connection per tab
Web Worker	Run the state projector (fold events into node states, H3 counts) off the main thread so the map never stutters
WebGL2 (via deck.gl / MapLibre)	GPU drawing of thousands of nodes, arcs and hexagons; WebGPU later, as library support matures
OffscreenCanvas	Draw heavy layers inside the worker, where the browser supports it
requestAnimationFrame	Batch all events that arrived in one frame into one update
Page Visibility API	Pause animations in hidden tabs and only apply the latest state when the tab comes back
IndexedDB	Cache the snapshot and recent events: instant first paint, offline replay mode

Service Worker + Web App Manifest	Installable PWA, works on weak networks, caches map tiles and the app shell
Push API + Notifications	Alert a provider to "new request, tap YES / NO" even when the tab is closed
Geolocation API	Requester's position, with permission; coarsen to an H3 cell before sending (§11)
BarcodeDetector + getUserMedia	Scan a provider's QR in the browser (Chromium-based browsers; use a library such as ZXing as fallback)
Web Crypto	Verify signed events in the browser before showing them
Custom Elements (web component)	`<grid-map src="/api/grid/stream" domain="ev">`: embed the Grid in any site, any framework

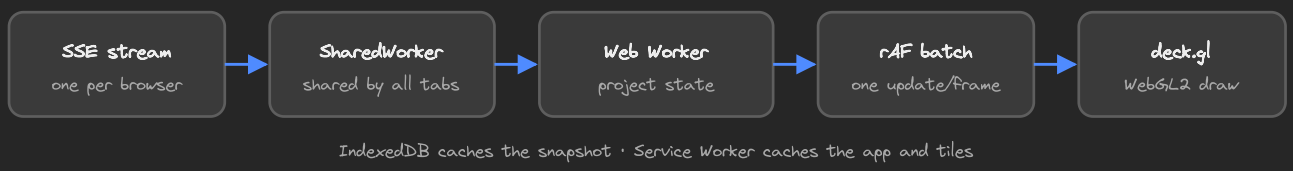


Figure 4. The browser pipeline: the main thread only draws.

keep people safe ↘

11. Privacy and security

Risk	Control
Requester's home location visible	Show requesters only as an H3 cell (res 7–8), never as a point. For stronger protection use geo-indistinguishability (Andrés et al., CCS 2013): add planar Laplace noise so nearby locations are statistically hard to tell apart.
Small providers' exact location	Providers choose public precision (exact for shops, cell-level for home workers and donors).
Fake events	Every event is signed with the agent's key; the Grid verifies against the registry before showing it.
Who can see what	Roles: public (counts, open/closed), session (the requester sees their own chain), provider (own node), admin (everything). Enforced in the push gateway with tokens.
Sensitive domains (health, blood)	Public view shows only aggregated H3 counts; identities appear only after both sides agree.
Data kept forever	Retention rules: raw events 30–90 days, aggregates longer. Make deletion of a node's history possible.

never break a deal ↘

12. Reliability and scale

- The Grid must never block a deal: agents emit events asynchronously (fire and forget into the bus). If the Grid is down, deals still happen.
- Idempotency: CloudEvents `id` lets the Grid drop duplicates when an agent retries.
- Ordering: order per session using a sequence number from the log; do not trust agents' clocks.
- Backpressure: if a viewer is slow, skip intermediate states and send the latest state per node.
- Heartbeats: nodes renew registration (like the Almanac); missing heartbeats turn the node grey automatically.

Stage	Nodes	Bus	Store	Transport
Demo	10–100	Node EventEmitter	SQLite	1 SSE route
One city pilot	1 000–10 000	Redis Streams or NATS	Postgres + PostGIS	SSE behind a proxy
Many cities	100 000+	NATS JetStream (subjects `grid.<city>.<domain>.*`) or Kafka	Postgres + ClickHouse	Gateway fleet + Redis pub/sub

same grid, new worlds ↘

13. One Grid, many domains

Because the Grid only understands nodes, rings and events, a new domain is a small config file, not new code.

```
# domains/blood.yaml
domain: blood
node_kind: donor
match: { blood_group: compatible(request.blood_group) }
rings_m: [1000, 3000, 10000]
want: 5
deadline_s: 300
human_confirm: true
trust_rules: [last_donation_over_90_days, verified_phone]
public_view: h3_counts_only        # privacy for donors
```

Domain	Node	Rings	Trust rule	Special view
Local shops	Shop agent	0.5 / 1 / 2 km	Hidden fees, closed	Prices in feed
Home services	Worker agent	2 / 5 / 10 km	Licence, rating	ETA arcs
Blood / health	Donor, hospital	1 / 3 / 10 km	Eligibility	Counts only
EV charging	Charger agent	1 / 3 / 8 km	Price cap, uptime	Free slots
Farm equipment	Owner agent	5 / 15 / 30 km	Deposit, ID	Village heatmap
Logistics	Truck agent	Route corridor	Permit, insurance	TripsLayer trails

honest check ↘

14. Is it new? Where the Grid stands

Existing	What it shows	What it does not do
Uber / Ola dispatch maps (H3)	Live supply and demand, matching	Closed, single company, no agent reasoning
Beckn / ONDC network	Open protocol for search → confirm	No standard live view of the network
Fetch.ai Agentverse / Almanac	Agent registry and discovery	No geographic, live view of deals
LangSmith, Langfuse, OTel traces	Step-by-step agent traces	Not geographic, not multi-party
Flightradar24, fleet trackers	Live positions of moving things	No negotiation, trust or deal flow

remember

Honest position: maps, registries, protocols and traces all exist. What is still rare is one open view that joins them: geographic discovery + agent-to-agent negotiation + trust verdicts + settlement, shown live and usable across domains. That join is the Grid's real contribution, and it becomes stronger if it is built on open standards (CloudEvents, H3, A2A / Beckn) instead of private formats.

from grid.py to platform ↘

15. Build plan, step by step

Each milestone is small enough to finish in a few days and keeps the demo working.

#	Milestone	Concrete changes	Done when
M1	Schema + ingest	Monorepo (pnpm + Turborepo): `packages/schema` with Zod CloudEvents types; `POST /api/events` in Next.js; Python agents call it with `httpx` instead of `grid.searched()` etc.; add the missing events	Every step of a deal appears in the feed
M2	Push + smooth map	SSE route with `Last-Event-ID`; React + MapLibre with markers kept in a `Map` by node ID and updated in place	No flicker, <300 ms delay
M3	Sessions + log	Drizzle table `events(seq, id, type, subject, time, data)` on SQLite then Postgres; projector per session; replay at `/map?replay=…`	Two buyers at once; restart loses nothing

M4	Spatial index	`h3-js` cell on every node and event; ripple uses Redis `GEOSEARCH` via `ioredis`; adaptive first ring	10 000 demo nodes, search <50 ms
M5	Browser pipeline	deck.gl layers (arcs, rings, H3 heatmap); projector in a Web Worker; SharedWorker stream; IndexedDB cache; stats bar	60 fps with 10 000 nodes on a mid-range laptop
M6	Out of process	NATS JetStream between ingest and gateways; events signed with agent keys and checked with Web Crypto	An agent on another laptop shows up on the Grid
M7	Privacy + roles + PWA	Auth (Auth.js or Clerk) with public / session / provider / admin views; H3-level requester locations; Service Worker, Push API for providers	Public map shows no personal data; app installs on a phone
M8	Domain configs + SDK	YAML per domain; `npm i @grid/sdk` (and a small Python client); `<grid-map>` web component built with Lit	A second domain runs with no Grid code change

What M1 looks like in code

```
// packages/schema/src/event.ts  (shared by server and browser)
import { z } from "zod";
export const GridEvent = z.object({
  specversion: z.literal("1.0"),
  id: z.string(), source: z.string(), subject: z.string(),
  type: z.string().startsWith("grid."), time: z.string().datetime(),
  data: z.record(z.unknown()),
});
export type GridEvent = z.infer<typeof GridEvent>;

// apps/web/app/api/events/route.ts  (ingest)
import { EventEmitter } from "node:events";
export const bus = new EventEmitter();                  // M6: swap for NATS JetStream

export async function POST(req: Request) {
  const ev = GridEvent.parse(await req.json());        // reject bad events early
  await db.insert(events).values(ev).onConflictDoNothing();   // idempotent on id (M3)
  bus.emit("event", ev);                                // SSE route listens to this
  return new Response(null, { status: 202 });
}

# directory.py (Python agent stays as it is, just posts the event)
httpx.post(GRID_URL + "/api/events", json={"specversion": "1.0", "id": str(uuid4()),
    "source": ctx.agent.address, "subject": f"session/{sid}", "type": "grid.ripple.expanded",
    "time": now_iso(), "data": {"lat": lat, "lng": lng, "radius_m": radius}})
```

for everyone ↘

16. Making it usable by everyone

For builders

- Open-source the Grid service and the event vocabulary
- Three-line SDK: `npm i @grid/sdk` then `grid.emit(...)`, plus a small Python client
- Drop-in `<grid-map>` web component for any site or framework
- Adapters: uAgents, A2A, Beckn callbacks → Grid events
- Docker one-command start with demo data

For providers

- Join by QR or WhatsApp; their node appears instantly
- Own "my node" view: requests, answers, earnings
- Choose how precise their location is

For requesters

- See only their own chain, in simple words
- Hindi and other Indian languages, voice summaries
- Works on cheap phones: low-data mode sends H3 counts, not maps

For cities and NGOs

- Public dashboards of gaps (where no provider exists)
- Emergency mode for blood, oxygen, shelters
- Export aggregated data (H3) for planning

in one line

One-line summary: the Grid is an open, standard event layer plus a live map for any network of agents. Agents register with an identity and a location, find each other in widening rings, negotiate with a contract-net style protocol, and emit CloudEvents; the Grid turns those events into a private, real-time, replayable view that works the same for shops, donors, chargers or trucks.

read more ↘

References

1. Uber Engineering, "H3: Uber's Hexagonal Hierarchical Spatial Index", uber.com/blog/h3 ; h3geo.org (resolutions, grid_disk)
2. Redis documentation, GEOSEARCH command, redis.io/docs/latest/commands/geosearch
3. PostGIS documentation, ST_DWithin, postgis.net
4. R. G. Smith, "The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver", IEEE Transactions on Computers, 1980
5. FIPA Contract Net Interaction Protocol Specification, fipa.org/specs/fipa00029
6. Beckn Protocol core specification (search, select, init, confirm, status, on_* callbacks), developers.becknprotocol.io
7. CloudEvents specification v1.0.2, cloudevents.io ; CNCF graduation announcement, cncf.io (25 January 2024)
8. OpenTelemetry semantic conventions for GenAI agent spans, opentelemetry.io/docs/specs/semconv/gen-ai/gen-ai-agent-spans
9. A2A (Agent2Agent) protocol and Agent Cards (/.well-known/agent-card.json), a2a-protocol.org (Linux Foundation)

10. Fetch.ai Almanac contract and registration, fetch.ai/docs/concepts/fetch-network/almanac
11. MDN Web Docs: EventSource, Web Workers, SharedWorker, BroadcastChannel, IndexedDB, Service Worker API, Push API, BarcodeDetector, Web Crypto, Custom Elements, developer.mozilla.org
12. Next.js route handlers and streaming, nextjs.org/docs ; h3-js, github.com/uber/h3-js ; ioredis, github.com/redis/ioredis ; Zod, zod.dev ; Drizzle ORM, orm.drizzle.team
13. deck.gl documentation (ScatterplotLayer, ArcLayer, TripsLayer, H3HexagonLayer), deck.gl ; MapLibre GL JS, maplibre.org
14. WHATWG, HTML Living Standard, Server-sent events (EventSource, Last-Event-ID), html.spec.whatwg.org
15. NATS JetStream documentation, docs.nats.io
16. M. E. Andrés, N. E. Bordenabe, K. Chatzikokolakis, C. Palamidessi, "Geo-Indistinguishability: Differential Privacy for Location-Based Systems", ACM CCS 2013, arXiv:1212.1984
17. Streetwise codebase: grid.py, directory.py, guardian.py, grahak.py, owner.py (Team SOLOX, Fetch-A-Thon 3.0)

Note: standards evolve. Status labels (e.g. OpenTelemetry GenAI conventions in Development) and version numbers were checked in October 2026 and should be re-checked before implementation.

← build it in small steps, keep the demo alive.